

Task 1

Files inside the folder

There are 2 files inside the dataset folder, apartments_for_rent_classified_10K and apartments_for_rent_classified_100K We selected the 10K dataset and used it for this task.

```
In [ ]: #importing of libraries
import pandas as pd
import numpy as np
import os
import kagglehub
#visualisations plotting&math
import matplotlib.pyplot as plt
import seaborn as sns

#machine learning imports
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.model_selection import train_test_split, RandomizedSearchCV, cross_val
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score, make

#models we chose
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

# download dataset from KaggleHub
path = kagglehub.dataset_download("shashanks1202/apartment-rent-data")
tenk_path = os.path.join(path, 'apartments_for_rent_classified_10K')

# Loading the CSV
df = pd.read_csv(
    os.path.join(tenk_path, "apartments_for_rent_classified_10K.csv"),
    sep=';',
    encoding='ISO-8859-1',
    engine='python',
    on_bad_lines='skip',
    quoting=3
)

# Clean text columns, remove newline & tab chars from text columns
df = df.map(lambda x: str(x).replace('\n', ' ').replace('\t', ' ') if isinstance(x,

#drop the 'price_display' column and ensure the and ensure target 'price' is numeri
```

```

if 'price_display' in df.columns:
    df.drop(columns=['price_display'], inplace=True)

df['price'] = pd.to_numeric(df['price'], errors='coerce')
df.dropna(subset=['price'], inplace=True)

# =====
# Feature Engineering

# Log-transform target to reduce skew
df['log_price'] = np.log1p(df['price'])

# Drop columns that don't help models directly
X = df.drop(columns=['price', 'log_price', 'id', 'title', 'body', 'time', 'source'])
y = df['log_price']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training set size:", X_train.shape)
print("Test set size:", X_test.shape)

# Define the custom transformer
class NewFeaturecreate(BaseEstimator, TransformerMixin):
    """
    Custom transformer to create a bathrooms-to-bedrooms ratio feature.
    A hyperparameter controls whether the feature is added.
    """
    def __init__(self, add_ratio_feature=True):
        self.add_ratio_feature = add_ratio_feature

    def fit(self, X, y=None):
        return self

    def transform(self, X, y=None):
        if self.add_ratio_feature:
            #ensure X is a DataFrame to access columns by name
            X_df = pd.DataFrame(X, columns=num_features)
            #prevent division by zero
            bedrooms = X_df['bedrooms'].replace(0, 1)
            ratio = X_df['bathrooms'] / bedrooms
            #add the new feature as a new column
            X_df['bathrooms_to_bedrooms'] = ratio
            return X_df
        else:
            return X

# =====
# PreProcessing
num_features = X_train.select_dtypes(include=['int64', 'float64']).columns
cat_features = X_train.select_dtypes(exclude=['int64', 'float64']).columns

```

```

numeric_transformer = Pipeline([
    ('feature_creator', NewFeaturecreate()),
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer([
    ('num', numeric_transformer, num_features),
    ('cat', categorical_transformer, cat_features)
])

# =====
# Define Models

models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(
        n_estimators=100, max_depth=20, n_jobs=-1, random_state=42
    )
}

# =====
# Training & Evaluation
results = {}
rmse_scorer = make_scorer(mean_squared_error, squared=False)

print("\n=== Cross-validation RMSE on Train Set ===")
for name, model in models.items():
    pipe = Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])

    # We use negative RMSE for scoring because cross_val_score maximizes the score
    scores = cross_val_score(pipe, X_train, y_train, cv=5, scoring='neg_root_mean_s
    train_rmse_cv = -np.mean(scores)

    print(f"{name}: {train_rmse_cv:.2f}")

print("\n=== Final Test Set Performance ===")
for name, model in models.items():
    pipe = Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)

    # Convert back from log scale
    y_pred_real = np.exp1(y_pred)

```

```
y_test_real = np.expml(y_test)

test_rmse = np.sqrt(mean_squared_error(y_test_real, y_pred_real))
mae = mean_absolute_error(y_test_real, y_pred_real)
r2 = r2_score(y_test_real, y_pred_real)

results[name] = {"Test RMSE": test_rmse, "MAE": mae, "R2": r2}

print(f"\n{name} Performance on test set:")
print(f"RMSE: {test_rmse:.2f}")
print(f"MAE: {mae:.2f}")
print(f"R2: {r2:.3f}")

# =====
# Picking Best Model

best_model_name = max(results, key=lambda k: results[k]['R2'])
print(f"\nBest Model: {best_model_name} with R2 = {results[best_model_name]['R2']:}.

best_model = Pipeline([
    ('preprocessor', preprocessor),
    ('model', models[best_model_name])
])
best_model.fit(X_train, y_train)

# =====
# Visualisation | Graphs and Plots

y_pred_best = np.expml(best_model.predict(X_test))
y_test_best = np.expml(y_test)

# Actual vs Predicted
plt.figure(figsize=(6,6))
sns.scatterplot(x=y_test_best, y=y_pred_best, alpha=0.5)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title(f"Actual vs Predicted Prices - {best_model_name}")
lims = [min(y_test_best.min(), y_pred_best.min()), max(y_test_best.max(), y_pred_be
plt.plot(lims, lims, 'r--')
plt.show()

# Residuals
residuals = y_test_best - y_pred_best
plt.figure(figsize=(6,4))
sns.histplot(residuals, kde=True)
plt.xlabel("Residual (Actual - Predicted)")
plt.title(f"Residual Distribution - {best_model_name}")
plt.show()

# Residuals vs Predicted
plt.figure(figsize=(6,4))
sns.scatterplot(x=y_pred_best, y=residuals, alpha=0.5)
plt.xlabel("Predicted Price")
plt.ylabel("Residual (Actual - Predicted)")
plt.title(f"Residuals vs Predicted Prices - {best_model_name}")
plt.axhline(y=0, color='r', linestyle='--')
```

```
plt.show()
```

Training set size: (8000, 15)

Test set size: (2000, 15)

=== Cross-validation RMSE on Train Set ===

Linear Regression: 0.36

Decision Tree: 0.29

Random Forest: 0.24

=== Final Test Set Performance ===

Linear Regression Performance on test set:

RMSE: 2148156024253.73

MAE: 48034229357.13

R²: -5187218368148190208.000

Decision Tree Performance on test set:

RMSE: 582.85

MAE: 298.58

R²: 0.618

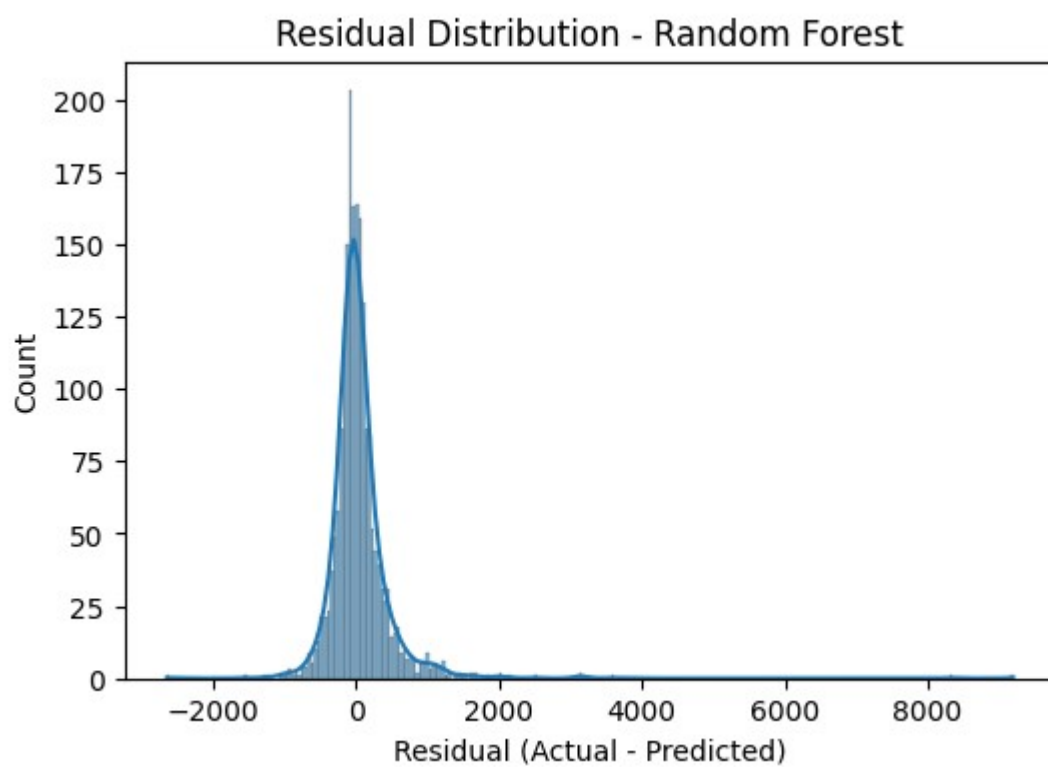
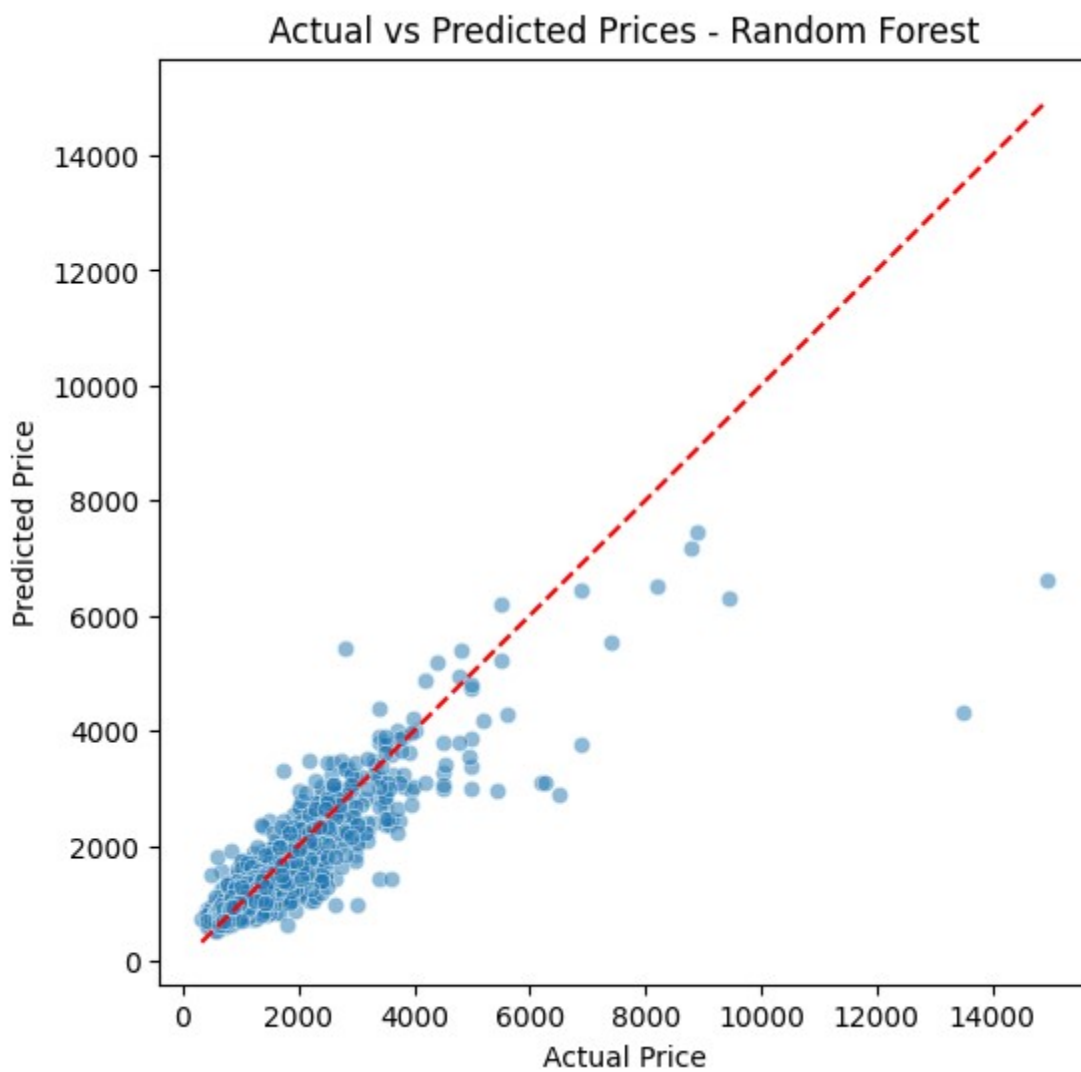
Random Forest Performance on test set:

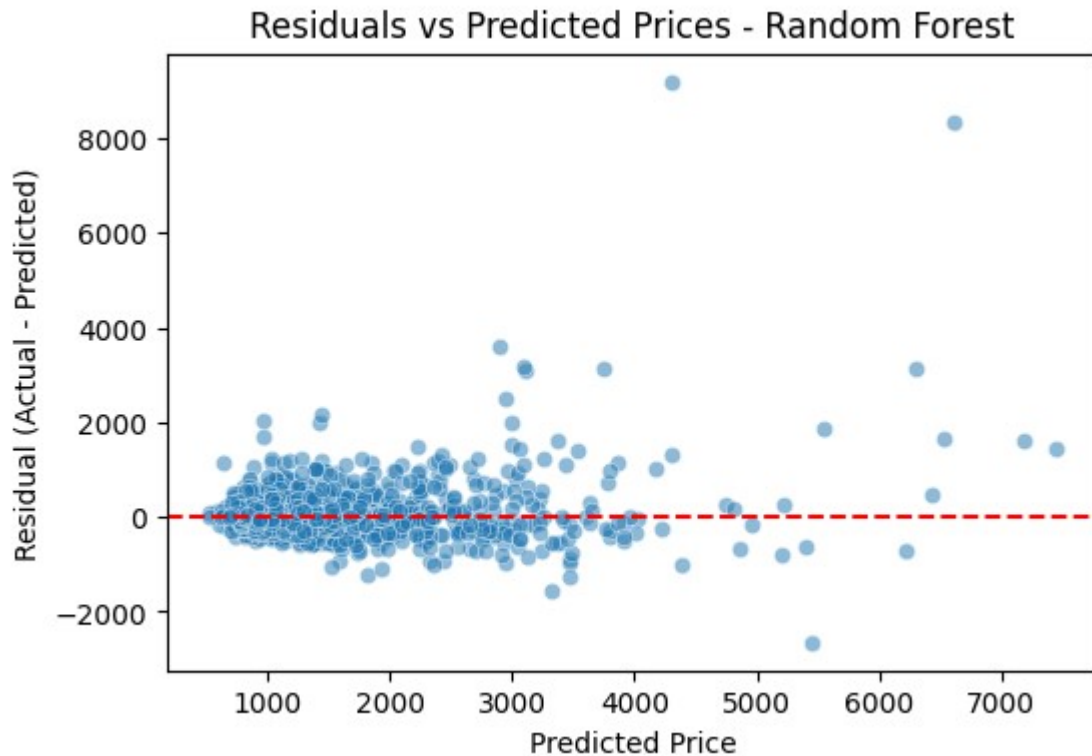
RMSE: 476.18

MAE: 243.08

R²: 0.745

Best Model: Random Forest with R² = 0.745





Overall code explanation for Task 1

1. We use the 'apartments_for_rent_classified_10K' dataset. The task was about predicting a target variable, using three machine learning algorithms. The algorithms we chose were:

Linear Regression

Decision Tree

Random Forest

2. The models and their metrics We chose 3 main metrics for this task. They are:

RMSE (Root Mean Squared Error)

MAE (Mean Absolute Error)

R^2 (Coefficient of Determination)

We did Cross-Validation on results of the training set which took quite sometime to calculate and produce the output. Here are the metrics for it:

Linear Regression: RMSE = 0.36

Decision Tree: RMSE = 0.29

Random Forest: RMSE = 0.24

Followed by our final test for the performance of each algorithm. Here the the metrics: Linear Regression:

RMSE: 2,148,156,024,253.73
(over two trillion!)

MAE: 48,034,229,357.13

R^2 : -5,187,218,368,148,190,208.000
(negative and extremely large-magnitude)

Decision Tree:

RMSE: 582.85

MAE: 298.58

R^2 : 0.618

Random Forest:

RMSE: 476.18

MAE: 243.08

R^2 : 0.745

3. Our Interpretation on Why Is There a Huge Problem With Linear Regression?

Linear Regression Catastrophe

The Linear Regression model's test performance is catastrophic as the RMSE and MAE are enormous, and the R^2 value is a hugely negative number. A valid R^2 is usually between 0 and 1 for a good model. Negative R^2 indicates the model performs dramatically worse than just predicting the mean.

Interpretation: This means that the Linear Regression model is not simply bad—it is way off the charts. These values says that this model is not suitable for this dataset.

Meanwhile, Decision Tree and Random Forest

Both Decision Tree and Random Forest have reasonable, relatively low RMSE/MAE and credible positive R^2 values more than 0.6, indicating relatively good fit on the dataset.

Random Forest outperforms Decision Tree here, as expected for a more robust ensemble

model.

These models behave as expected on the dataset unlike Linear Regression.

4. Best Model for this dataset

Random Forest is the best model here with RMSE: 476.18, MAE: 243.08, R^2 : 0.745 , on the test set. This model is robust to outliers and poorly scaled data, unlike Linear Regression. Hence, Random Forest is the best Model.

5. Visualstions of the best Model

There are visualtions of the model on how it perfomed on the dataset in the output above.